

## GPU VS CPU DESIGN PHILOSOPHY

| Aspect            | CPU (Latency-Opt)     | GPU (Throughput-Opt)   |
|-------------------|-----------------------|------------------------|
| Primary goal      | Min single-op latency | Max total throughput   |
| Cores             | 2-64 powerful         | 1000s simple           |
| ALU               | Complex, OoO          | Simple, in-order       |
| Cache             | Large L1/L2/L3        | Small L1, shared mem   |
| Branch pred.      | Advanced speculative  | None/minimal           |
| Data fwd.         | Yes, sophisticated    | No or minimal          |
| Execution         | Superscalar, OoO      | SIMD in-order          |
| Clock             | High (3-5 GHz)        | Lower (1-2 GHz)        |
| Best for          | Serial, low-lat tasks | Parallel, high-thruput |
| Thread ctx switch | Expensive (µs)        | Zero cost (cycles)     |

**Why specialize?** Moore's Law slowed (quantum limits, heat). Dennard scaling ended (power < density). CPU optimization wall hit → domain-specific accelerators win.

| Device | Specialization       | Use case                 |
|--------|----------------------|--------------------------|
| GPU    | Parallel throughput  | Graphics, AI/ML, HPC     |
| TPU    | Matrix multiply      | Neural network inference |
| ASIC   | Fixed function       | Bitcoin mining           |
| FPGA   | Reconfigurable logic | Flexible workloads       |

**GPU latency hiding:** When warp stalls on memory (100-200cy latency), scheduler instantly switches to another ready warp. Need many active warps to hide latency. CPUs use branch prediction + large caches instead.

## COMPUTER ARCHITECTURE REVIEW

## CPU Memory Hierarchy:

| Level     | Size    | Latency | Purpose    |
|-----------|---------|---------|------------|
| Registers | ~1 KB   | ~0.5 ns | Operands   |
| L1 Cache  | 32 KB   | ~4 ns   | Hot data   |
| L2 Cache  | 256 KB  | ~11 ns  | Per-core   |
| L3 Cache  | 8-20 MB | ~40 ns  | Shared     |
| DRAM      | GBs     | ~100 ns | Main mem   |
| Disk/SSD  | TBs     | ms-µs   | Persistent |

**Virtual Memory:** Each process sees its own isolated address space (256 TB on 64-bit). OS+MMU translate virtual → physical via **page tables**. Pages typically 4 KB. If page not in RAM → page fault → OS fetches from disk (very slow).

## Process vs Thread:

- **Process:** Own virtual address space, file descriptors, heap. Context switch = expensive (TLB flush, cache miss)
- **Thread:** Shares address space + heap with other threads. Private stack, registers, PC. Faster context switch.

**ISA (Instruction Set Architecture):** Contract between HW and SW. Defines instruction encoding, register set, memory model. GPU equivalent: PTX (virtual ISA). MIPS R-format: [op:6][rs:5][rt:5][rd:5][shamt:5][funct:6]

## CUDA FUNCTION QUALIFIERS

| Qualifier                        | Runs on      | Called from | Notes                         |
|----------------------------------|--------------|-------------|-------------------------------|
| <code>__global__</code>          | Device (GPU) | Host (CPU)  | Kernel; must return void      |
| <code>__device__</code>          | Device       | Device only | Helper/utility function       |
| <code>__host__</code>            | Host (CPU)   | Host only   | Normal C; optional tag        |
| <code>__host__ __device__</code> | Both         | Both        | Compiled for both (templates) |

## Kernel Launch Syntax:

```
kernel<<<gridDim, blockDim, sharedMem, stream>>>(args);
// sharedMem = dynamic shared mem bytes (default 0)
// stream = CUDA stream (default 0)
```

**Key Constraint:** `__global__` functions must return `void`. Cannot use function pointers to device functions from host.

## CUDA APIS — COMPLETE REFERENCE

## Memory Allocation:

```
cudaMalloc(void** devPtr, size_t size); // GPU DRAM
cudaFree(void* devPtr);
cudaHostAlloc(&h_ptr, size, cudaHostAllocDefault); // pinned
cudaFreeHost(h_ptr);
cudaMemset(devPtr, val, size); // zero GPU mem
cudaMallocManaged(&ptr, size); // unified memory
```

## Data Transfer:

```
cudaMemcpy(dst, src, sz, cudaMemcpyHostToDevice); // H->D sync
cudaMemcpy(dst, src, sz, cudaMemcpyDeviceToHost); // D->H sync
cudaMemcpy(dst, src, sz, cudaMemcpyDeviceToDevice); // D->D sync
cudaMemcpyAsync(dst, src, sz, dir, stream); // non-blocking!
```

## Synchronization:

```
cudaDeviceSynchronize(); // CPU blocks until ALL GPU work done
__syncthreads(); // barrier: ALL threads in block sync here
```

## CUDA PARALLELISM: GRID → BLOCK → THREAD

## 3-Level Hierarchy:

- **Grid:** All blocks for one kernel launch. Blocks run independently in any order. 1D/2D/3D. `gridDim` = grid shape.
- **Block:** Group sharing `__shared__` mem + can `__syncthreads()`. Max **1024 threads/block**. All threads must fit on one SM. `blockDim` = block shape.
- **Thread:** Individual unit. Private registers + local mem. `threadIdx` = position in block.

**Built-in Variables:** `threadIdx.{x,y,z}` · `blockIdx.{x,y,z}` · `blockDim.{x,y,z}` · `gridDim.{x,y,z}`

## Index Formulas:

```
// 1D kernel
int i = threadIdx.x + blockDim.x * blockIdx.x;

// 2D kernel (image: row-major)
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
int idx = row * width + col; // - row-major layout

// 3D kernel
int x = blockIdx.x*blockDim.x + threadIdx.x;
int y = blockIdx.y*blockDim.y + threadIdx.y;
int z = blockIdx.z*blockDim.z + threadIdx.z;
int idx = z*(Dy*Dx) + y*Dx + x;
```

## Grid/Block Configuration:

```
// 1D: n elements, 256 threads/block
dim3 block(256);
dim3 grid((n + 255) / 256); // ceiling division
```

```
// 2D: WxH image, 16x16 blocks
dim3 block(16, 16);
dim3 grid((W+15)/16, (H+15)/16);

// Boundary check in kernel (ALWAYS needed when grid overestimates)
if (i < n) C[i] = A[i] + B[i]; // guard for extra threads
```

## SM Occupancy Rules (exam-critical):

- SM holds limited threads (e.g. 1536) AND limited blocks (e.g. 8)
- Bottleneck = whichever limit hits first
- 256 threads/block → 6 blocks × 256 = 1536 = fills SM exactly ✓
- 64 threads/block → capped at 8 blocks = 512 (wastes 1024 thread slots)
- 1024 threads/block → 1 block fits (1536 threads left but <1024 remain)

## SIMT EXECUTION MODEL &amp; WARP DIVERGENCE

**Warp Basics:** 32 consecutive threads execute in lockstep on SIMD HW. Threads 0-31 = warp 0, 32-63 = warp 1, etc. All threads in a warp share ONE program counter. AMD equivalent: "wavefront" (64 threads).

## Warp Divergence — Full Explanation:

- Threads in same warp take different branches → HW *serializes* them using **predication**
- Step 1: Execute "taken" path, mask off non-taken threads (they idle)
- Step 2: Execute "not-taken" path, mask off taken threads (they idle)
- Result: Warp time = time(path A) + time(path B). Worst case: 2× slowdown for if/else
- With N distinct paths: N× slowdown

```
// DIVERGES (threadIdx varies within warp 0: threads 0-31)
if (threadIdx.x < 16) { pathA(); } else { pathB(); }
```

```
// NO DIVERGE (blockIdx same for all threads in block)
if (blockIdx.x < 4) { pathA(); } else { pathB(); }
```

```
// NO DIVERGE (blockDim constant per kernel)
if (blockDim.x > 4) { pathA(); }
```

```
// DIVERGES - causes warp divergence:
if (threadIdx.x > 4) // splits warp 0 at position 4
```

## Divergence Counting Method (for reduction if(tid%stride==0)):

- Active threads = those with `tid % stride == 0` → count = `blockDim/stride`
- A warp diverges if it has BOTH active and inactive threads
- Map active tids to warps (warp = `tid/32` integer division)
- stride=1: all active → 0 div warps
- stride=32: active at 0,32,64,... → 1 per warp → all 64 warps diverge
- stride=128: active at 0,128,256,... → 1 per 4 warps → 16 warps diverge
- stride=blockDim: only 1 active thread in whole block → 1 warp diverges

## Efficient Reduction vs Basic (exam pattern):

```
// BASIC (divergent) - scattered active threads
if (tid % (2*s) == 0) → diverges every warp
```

```
// EFFICIENT (non-divergent) - consecutive active threads
if (tid < s) // only boundary warp diverges
    s_data[tid] += s_data[tid + s]; // converging stride
```

**SIMT Stack:** Each warp has a hardware stack tracking divergent paths. Push on branch-with-divergence, pop when paths reconverge. **Reconvergence point** = first instruction after divergent if/else.

## CUDA MEMORY HIERARCHY — FULL DETAIL

| Type       | Scope       | Lifetime | Speed        | Size         | Decl                         |
|------------|-------------|----------|--------------|--------------|------------------------------|
| Registers  | Thread      | Thread   | ~0 cy        | ~256KB/SM    | auto vars                    |
| Local Mem  | Thread      | Thread   | DRAM slow    | Limited      | arrays in thread             |
| Shared Mem | Block       | Block    | ~1-4 cy      | 48-96KB/blkl | <code>__shared__</code>      |
| L1 Cache   | Block       | Kernel   | ~20 cy       | 32-128KB     | automatic                    |
| L2 Cache   | Device      | Kernel   | ~80 cy       | MB           | automatic                    |
| Global Mem | All threads | App      | ~200 cy      | GBs          | <code>__device__</code>      |
| Constant   | All (R/O)   | App      | Fast if same | 64 KB        | <code>__constant__</code>    |
| Texture    | All (R/O)   | App      | Cached       | -            | <code>texture&lt;&gt;</code> |

**Registers — Register Pressure:** Compiler automatically maps local vars to registers. If kernel needs too many registers → **register spilling** to local memory (DRAM speed). Fewer regs/thread = more threads per SM = better occupancy. Control with `__launch_bounds__(maxThreads)` or `-maxrregcount=N`.

## Shared Memory — Usage Pattern:

```
_global_ void kernel(float* A, int n) {
    __shared__ float s[256]; // allocated per block
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + tid;

    s[tid] = (gid < n) ? A[gid] : 0.0f; // load
    __syncthreads(); // BARRIER - all must reach here

    // Now safe to read any s[i] from any thread
    float result = s[(tid*1) % 256];
    __syncthreads(); // before next modification
}
```

**Dynamic shared mem:** extern `__shared__ float s[]`; and pass bytes at launch: `kernel<<<g,b,bytes>>>());`

## Constant Memory:

```
_device_ __constant_ float c_filter[16];
// Host: cudaMemcpyToSymbol(c_filter, h_filter,
16*sizeof(float));
// Device: reads c_filter[i] - broadcast cache (fast if all threads read same index)
```

**Global Memory Bandwidth:** Modern GPUs: ~1-2 TB/s (e.g. A100: 2 TB/s). PCIe bandwidth: ~32 GB/s (64× slower). This is why minimizing H → D transfers matters.

## MEMORY COALESCING &amp; OPTIMIZATION

**Coalescing Rule:** GPU memory hardware coalesces consecutive-thread accesses to consecutive addresses into ONE 128-byte cache line transaction. Non-coalesced = 32 separate transactions = 32× BW waste.

```
// COALESCED ✓ - thread i reads arr[i]
float val = arr[threadIdx.x + blockDim.x*blockIdx.x];
// Threads 0-31: read addresses 0,4,8,...,124 → 1 transaction
```

```
// NOT COALESCED ✗ - stride-32 pattern
float val = arr[(threadIdx.x + blockDim.x*blockIdx.x) * 32];
// Threads 0-31: read 0, 128, 256, ... → 32 transactions
```

```
// 2D ARRAY - use row-major access (C order)
float val = A[row * WIDTH + col]; // ✓ cols are consecutive
float val = A[col * HEIGHT + row]; // ✗ rows are strided (column-major)
```

## Tiling/Blocking for Shared Memory:

```
// Matrix multiply tile pattern
_global_ void matmul(float* A, float* B, float* C, int N) {
    __shared__ float sA[TILE][TILE], sB[TILE][TILE];
    int row = blockIdx.y*TILE + threadIdx.y;
    int col = blockIdx.x*TILE + threadIdx.x;
    float sum = 0.0f;
    for(int t = 0; t < N/TILE; t++) {
        sA[tid][tx] = A[row*N + (t*TILE+tx)]; // load tile
        sB[tid][tx] = B[(t*TILE+ty)*N + col];
        __syncthreads(); // all loaded
        for(int k=0; k<TILE; k++) sum += sA[tid][k]*sB[k][tx];
        __syncthreads(); // before next tile
    }
    C[row*N+col] = sum;
}
```

Speedup: T× fewer global loads (each element reused T times by T threads in tile).

**Bank Conflicts:** Shared mem has 32 banks (4B each). Bank# = `addr/4 % 32`. Two threads same bank same cycle → serialized.

```
// CONFLICT: stride-2 access
s[2*tid] // banks: 0,2,4,6,... → every warp uses 16 banks ✓ OK
s[32*tid] // banks: 0,0,0,... → ALL threads hit bank 0 → 32-way conflict!
// FIX: pad array
__shared__ float s[32 + 1]; // padding shifts bank assignments
```

## PARALLEL PATTERNS — REDUCTION, SCAN, STENCIL

## Parallel Reduction (sum N → 1):

```
// Two loading strategies:
// A) Adjacent pairs: thread t loads [2t] and [2t+1]
int start = 2*blockIdx.x*blockDim.x;
int t = 2*threadIdx.x; // local shared index
pSum[t] = input[start + t];
pSum[t+1] = input[start + t + 1];
```

```
// B) Strided: thread t loads [t] and [blockDim+t]
int t = threadIdx.x; // local shared index
pSum[t] = input[start + t];
pSum[blockDim.x+t] = input[start + blockDim.x + t];
// B is BETTER: coalesced loads! All threads read consecutive adds.
```

```
// Reduction loop (efficient, converging):
for(int s = blockDim.x/2; s > 0; s /= 2) {
    if(tid < s) pSum[tid] += pSum[tid + s];
    __syncthreads();
}
```

// Only 1 divergent warp (boundary at s=1)

**Complexity:** O(log n) steps, O(n) total work. Requires multi-pass for arrays larger than one block.

## Prefix Sum / Scan: output[i] = sum(input[0..i])

```
// Blelloch (work-efficient): O(n) work, O(log n) time
// Up-sweep (reduce) phase:
for(d=1; d<n; d*=2) { if fits: s[k] += s[k - d]; }
// Down-sweep (distribute) phase:
s[n-1]=0;
for(d=n/2; d>0; d/=2) { swap + add at distance d }
```

**Stencil / Convolution:** Each output depends on neighborhood of inputs (e.g., blur, finite diff).

```
// Load halo (ghost) cells into shared memory
__shared__ float s[BLOCK + 2*HALO];
// Interior threads: each loads 1 element
s[HALO + tid] = input[tid];
// Boundary threads: also load halo elements
if(tid < HALO) {
    s[tid] = input[gid - HALO]; // left halo
    s[HALO + BLOCK + tid] = input[gid + BLOCK]; // right
}
__syncthreads();
// Compute using fast shared memory
```

## OCCUPANCY, ATOMICS &amp; ADVANCED

**Occupancy = active warps / max warps per SM**

| Factor                  | Effect                   | Fix                            |
|-------------------------|--------------------------|--------------------------------|
| Registers/thread ↑      | Fewer threads/SM         | Reduce vars, use -maxrregcount |
| Shared mem/block ↑      | Fewer blocks/SM          | Reduce tile size               |
| Threads/block too small | Many blocks, few threads | Increase block size            |
| Threads/block too large | Fits fewer blocks        | Reduce block size              |

**Example:** SM has 1536 thread limit, 8 block limit. Kernel uses 32 regs/thread, 4KB shared/block. At 256 threads/block: threads limit → 6 blocks. Shared limit → 8+ blocks possible. Thread limit binds → 6 blocks × 256 = 1536 threads. Occupancy = 1536/1536 = 100%.

**Atomic Operations:** Safe concurrent read-modify-write to same address. Expensive (serialized at memory).

```
atomicAdd(&var, val); // var += val (returns old val)
atomicSub(&var, val); // var -= val
atomicExch(&var, val); // var = val, return old
atomicMax(&var, val); // var = max(var, val)
atomicCAS(&var, cmp, val); // if(var==cmp) var=val; return old
// 10-100× slower than normal store. Use sparingly.
```

## Intrinsic Functions (fast approximations):

```
_sinf(x) // fast sin (~2 ULP error)
_expf(x) // fast exp
_fdividef(x,y) // fast divide
_popc1l(x) // count set bits
// Use -use_fast_math to auto-replace math.h calls
```

## Debugging Tools:

```
cuda-memcheck ./prog // mem errors, leaks, races
cuda-gdb ./prog // GDB for device code (compile -G)
// In kernel:
printf("tid=%d val=%f\n", threadIdx.x, val); // works!
```

## PERFORMANCE ANALYSIS &amp; PROFILING

**Nsight Systems (nsys):** System-level. Timeline view of CPU/GPU activity, PCIe transfers, kernel launches.

nsys profile --stats=true ./my\_program

**Nsight Compute:** Kernel-level deep dive. Measures SM utilization, warp efficiency, memory BW utilization, cache hit rates, instruction throughput, roofline.

nv-nsight-compute-cli --set full ./my\_program

```
blocks!)
```

```
    // (device code only; NOT across
```

**Device Info:**

```
cudaDeviceProp prop;
cudaGetDeviceProperties(&, device_id);
// prop.maxThreadsPerBlock, prop.sharedMemPerBlock, etc.
```

**Error Checking:**

```
cudaError_t err = cudaGetLastError();
if(err != cudaSuccess)
    printf("%s\n", cudaGetErrorString(err));
```

### NVCC COMPILER & PTX

**Compilation Pipeline:**

```
foo.cu → NVCC → { device code → PTX → CUBIN (machine code)
                  { host code → g++/clang → host obj }
                  → link → executable
```

**PTX (Parallel Thread eXecution):** Virtual ISA — architecture-independent intermediate representation. Driver JIT-compiles PTX → actual GPU instructions at runtime. Benefits: binary compatibility across GPU generations; unlimited virtual registers; new GPUs just need new JIT backend.

**Key NVCC Flags:**

| Flag             | Effect                               |
|------------------|--------------------------------------|
| -G               | Device debug symbols (disables opts) |
| -g               | Host debug symbols                   |
| -O3              | Enable device optimizations          |
| -lineinfo        | Source lines for profiler            |
| -arch=sm_86      | Compile for GPU compute cap 8.6      |
| -x cu            | Treat .cc files as .cu               |
| -Xcompiler -flag | Pass flag to host compiler           |
| --ptx            | Output PTX only (inspect IR)         |

```
nvcc -G -Xcompiler -g prog.cu      # debug build
nvcc -O3 -lineinfo prog.cu        # profiling build
nvcc -arch=sm_86 -O3 prog.cu      # target specific GPU
```

### CUDA STREAMS & ASYNC EXECUTION

**Stream = FIFO queue of GPU operations.** In-stream: sequential. Cross-stream: can overlap (concurrent copy + compute).

```
cudaStream_t s1, s2;
cudaStreamCreate(&s1); cudaStreamCreate(&s2);

// Overlap copy-to-device with kernel on different data
cudaMemcpyAsync(d_A, h_A, sz, H2D, s1);
kernel<<<grid,block,0,s2>>>(d_B); // concurrent!

cudaStreamSynchronize(s1); // wait for s1 only
cudaStreamSynchronize(s2);
cudaStreamDestroy(s1); cudaStreamDestroy(s2);
```

**Pinned Memory:** Required for async transfers. cudaHostAlloc allocates page-locked host RAM. DMA engine can transfer directly (no CPU copy to temp buffer). Faster H → D bandwidth.

```
float* h_data;
cudaHostAlloc(&h_data, size, cudaHostAllocDefault); // pinned
// Now cudaMemcpyAsync works without extra copy
cudaFreeHost(h_data); // not free()!
```

**Events (stream operations):** Kernel launches + memory copies = CUDA events. NOT all CUDA API calls. Events processed in FIFO within stream; cannot be out-of-order.

**Single stream = NO concurrent copy+compute.** Need ≥2 streams AND pinned memory for overlap.

### Key Metrics to Watch:

| Metric             | Good | Means if low                     |
|--------------------|------|----------------------------------|
| SM Active Cycles % | >80% | Kernel too small / too few warps |
| Warp Efficiency    | >90% | Divergence, imbalanced work      |
| Mem BW utilization | >70% | Compute bound (may be OK)        |
| L2 Hit Rate        | >50% | Poor data reuse, no tiling       |
| Occupancy          | >50% | Register/shared mem pressure     |

**Common Pitfalls Summary:**

- **Global mem per op:** High latency stalls. Fix: `__shared__` + tiling
- **Warp divergence:** Idle threads. Fix: reorganize control flow
- **Uncoalesced access:** 32× BW waste. Fix: consecutive thread → addr mapping
- **Bank conflicts:** Serialized shared reads. Fix: padding `[N+1]`
- **Missing \_\_syncthreads:** Race + stale data. Always sync after shared writes
- **PCIe bottleneck:** Use pinned mem, overlap transfers, minimize H → D
- **Register spill:** Reduces occupancy. Reduce register count per thread
- **cudaMalloc in loop:** Very slow. Allocate once, reuse

### QUICK MC / T-F ANSWER KEY

| Statement / Question                                                  | Answer                              |
|-----------------------------------------------------------------------|-------------------------------------|
| DMA transfers between <code>__</code>                                 | Physical addresses                  |
| <code>cudaMalloc</code> allocates in                                  | Device (GPU) memory                 |
| <code>cudaHostAlloc</code> allocates in                               | Pinned (page-locked) host memory    |
| Single stream: concurrent copy + kernel?                              | NO — streams serialize              |
| CUDA stream is a <code>__</code>                                      | Queue (FIFO)                        |
| Stream commands out-of-order?                                         | FALSE                               |
| All CUDA API calls are CUDA events?                                   | FALSE (only kernels + copies)       |
| Virtual → physical via page tables?                                   | TRUE                                |
| All warps execute same instr simultaneously?                          | FALSE (only within a warp)          |
| Warps finish at different times?                                      | TRUE                                |
| GPU warp scheduler exists?                                            | TRUE                                |
| PTX = intermediate GPU representation?                                | TRUE                                |
| GPUs can boot an OS?                                                  | FALSE                               |
| <code>if(threadIdx.x &gt; 4) → divergence?</code>                     | YES (splits warp 0)                 |
| <code>if(blockDim.x &gt; 4) → divergence?</code>                      | NO (constant per warp)              |
| <code>if(blockIdx.x &gt; 4) → divergence?</code>                      | NO (same for whole block)           |
| SM: 1536 thread, 8 block limit. Best 1D block config for max threads? | 256/block (6 blocks × 256 = 1536)   |
| <code>__syncthreads()</code> syncs across blocks?                     | FALSE — within block only           |
| <code>cudaMalloc</code> zeroes memory?                                | FALSE — use <code>cudaMemset</code> |
| OS can swap out pinned memory?                                        | FALSE — page-locked                 |

## OFFICIAL MIDTERM QUESTIONS — FULLY SOLVED

## Q1 [10 pts] Warp Divergence Counting in Reduction

Block=2048 threads, warp size=32. Code: `if(threadIdx.x % stride == 0) { pSum[2*tid] += pSum[2*tid+stride]; }`

How many warps diverge at stride=1? stride=32? stride=128?

**FULL REASONING:** Total warps = 2048/32 = **64 warps**. Warp W = threads [32W ... 32W+31]. A warp diverges iff it has BOTH active and inactive threads (mixed).

**stride=1:** Condition `tid % 1 == 0` is true for ALL threads → 2048 active, 0 inactive in every warp → no divergence. **Answer: 0**  
**stride=32:** Active threads: {0, 32, 64, 96, ...} = multiples of 32. In warp W, only thread 32W is active, threads 32W+1..32W+31 are inactive. Every warp has 1 active + 31 inactive → all 64 warps diverge. **Answer: 64**  
**stride=128:** Active threads: {0, 128, 256, ...} = multiples of 128. In warp W: active iff 32W is a multiple of 128, i.e., W is a multiple of 4. Warps 0,4,8,...,60 each have 1 active (diverge) = 16 warps. Warps 1,2,3,5,... are fully inactive (no divergence). **Answer: 16**  
**Pattern: divergent warps = 2048/(stride\*32) when stride=32, capped at 64.**

## Q2 [20 pts] Reduction Load — Adjacent Pairs

Each thread loads 2 adjacent elements: thread 0 → [0],[1]; thread 1 → [2],[3].

Fill in: `uint t = ____; pSum[t] = input[start+____]; pSum[t+1] = input[start+____];`

**SOLUTION:** Thread i must own shared mem slots [2i, 2i+1] and global positions [start+2i, start+2i+1].

```
unsigned int t = 2 * threadIdx.x;    // t=0,2,4,6,...
partialSum[t] = input[start + t];    // start+0,2,4,...
partialSum[t+1] = input[start + t+1]; // start+1,3,5,...
```

**WHY** `t=2*threadIdx.x`: Thread 0 occupies shared slots 0,1. Thread 1 occupies 2,3. Pattern: slot = 2\*thread\_id. Global offset mirrors shared index since start already block-offsets us. Access pattern: threads load pairs (0,1),(2,3),(4,5),... — NOT coalesced (each thread reads every 2 words).

## Q3 [20 pts] Reduction Load — Stride = blockDim

Thread 0 loads [0] and [blockDim]; thread 1 loads [1] and [blockDim+1].

Fill in: `uint t = ____; pSum[t] = input[start+____]; pSum[blockDim.x+t] = input[start+____];`

**SOLUTION:** Thread i owns shared slot [i] and [blockDim+i], and global positions [start+i] and [start+blockDim+i].

```
unsigned int t = threadIdx.x;    // t=0,1,2,...
partialSum[t] = input[start + t];
partialSum[blockDim.x + t] = input[start + blockDim.x + t];
```

**WHY** this is BETTER than Q2: All threads access consecutive memory addresses (t=0,1,2,...). Two coalesced transactions load entire 2\*blockDim region. Q2 was non-coalesced (stride=2 access from global). This layout also works better for reduction since first half [0..B-1] holds first elements, second half [B..2B-1] holds second elements.

## Q4 [10 pts] Image Kernel: Grid/Block Dims &amp; Divergence

Image 400×900 pixels, 1 thread/pixel, square blocks, max 1536 threads/SM, max 8 blocks/SM.

a) Best block & grid dimensions? b) With 16×16 blocks: how many warps diverge?

**a) Block selection analysis:**

| Block size | Threads | Blocks fitting (thread lim) | Blocks fitting (block lim) | Active threads   |
|------------|---------|-----------------------------|----------------------------|------------------|
| 8×8=64     | 64      | 1536/64=24                  | 8                          | 8×64=512         |
| 16×16=256  | 256     | 1536/256=6                  | 6 (under 8)                | 6×256=1536 ✓ MAX |
| 32×32=1024 | 1024    | 1536/1024=1                 | 1                          | 1×1024=1024      |

**Best: 16×16 blocks.** Grid = [400/16] × [900/16] = **25 × 57 blocks**

**b) Divergence calculation:** Width=900, blocks\_x=57. 57th col block covers cols 896–911 but image only goes to 899 → 4 valid, 12 invalid per row. Each block has 16 rows of 16 threads. Warps: each 32-thread warp spans 2 rows of 16. So 8 warps per block. All 8 warps in an overhang block diverge (some threads check col<900 and branch).  
 Height=400, 400/16=25.0 exactly → no vertical overhang. Only right edge.

Overhang blocks: 25 rows × 1 overhang col = 25 blocks × 8 warps = **200 divergent warps**

## OFFICIAL QUESTIONS CONTINUED

## Q5 [20 pts] Broken VecAdd — Full Analysis

Block=256, nBlocks=(n-1)/256+1. Kernel loops over ALL elements (no threadIdx indexing):

```
__global__ void VecAdd(int n, float*A, float*B, float*C){ for(uint i=0;i<n;i++){
  C[i]=A[i]+B[i]; }
```

**a) Correct results? YES.** Every thread independently computes `ALL C[i] = A[i]+B[i]`. Since A and B are read-only and every thread writes the same correct value to each `C[i]`, no race condition on output values. Final `C[i]` is correct but massively redundant.

**b) Additions?** Let `nB = [n/256] = (n-1)/256+1`. Threads total = 256 × nB. Each thread does n additions (the loop).

**This VecAdd: 256 × nB × n additions**

Lab 1 correct VecAdd (one element per thread, no loop): **n additions total**.

Ratio: 256 × nB = 256 × (n/256) = n times MORE work than needed. Catastrophically inefficient.

**c) Memory loads?** Each iteration loads `A[i]` and `B[i]` = 2 loads. n iterations × 256×nB threads.

**This VecAdd: 2 × 256 × nB × n loads. Lab 1: 2n loads.**

**d) Serial vs parallel? Serial CPU is faster here.** Both produce correct results in O(n) real work, but GPU does O(n²) total work AND incurs overhead: `cudaMalloc`, H → D transfer, kernel launch, D → H transfer, `cudaFree`. The GPU "parallel" version doesn't parallelize at all — every thread reruns the entire serial loop. No speedup from parallelism.

## Q6 [20 pts] Performance Issues — Global Mem &amp; Warp Divergence

**a) Global memory access per operation — harm & fix:**

**Harm:** Global memory latency = ~200 cycles. If every arithmetic op requires a global load, the kernel stalls waiting for data every instruction. With 32 threads in a warp stalling simultaneously, SM is idle. Throughput collapses to memory bandwidth / latency (arithmetic units starved). Creates memory-bound bottleneck even if compute units are available.

**Fix:** Use `__shared__` memory as user-managed cache. Load a tile of data once (cooperative loading by all threads in block), then perform all operations on fast shared memory. This reduces global bandwidth by the reuse factor (tile size). Example: matrix multiply with tiling reduces global loads from 2N² to 2NVT.

**b) Warp divergence — harm & fix:**

**Harm:** All 32 threads in a warp share one instruction pipeline. When an if/else depends on thread data, HW must execute both paths with masking (SIMT stack). Threads on the "wrong" path are idle — compute units wasted. In worst case (every thread takes unique path), throughput = 1/32 of peak. Serialization within a warp.

**Fix:** Reorganize thread indexing so threads in same warp always take same path. In reduction: change `if(tid%(2s)==0)` (divergent, scattered active threads) to `if(tid<s)` (non-divergent, consecutive active threads). Result: all active threads in first / 32 warps; remaining warps fully idle; only boundary warp potentially diverges.

## Q7 MC: Streams, Memory, DMA

| Q                                      | Answer           | Explanation                                                                                                                                        |
|----------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| DMA transfers between ____             | Physical addr    | DMA bypasses MMU/virtual mem; needs physical addresses; pinned mem prevents OS from moving the physical page                                       |
| cudaMalloc allocs in ____              | Device memory    | Allocates GPU DRAM (not host RAM, not shared, not pinned)                                                                                          |
| cudaHostAlloc allocs in ____           | Pinned memory    | Page-locked host RAM. Enables DMA without intermediate copy. NOT device memory.                                                                    |
| Single stream: concurrent copy+kernel? | NO               | FIFO queue: copy must finish before kernel starts in same stream                                                                                   |
| Stream is a ____                       | Queue            | FIFO ordered sequence of GPU operations                                                                                                            |
| Commands out-of-order in stream?       | FALSE            | Within stream: strict FIFO. Commands across different streams CAN overlap.                                                                         |
| All CUDA API calls are events?         | FALSE (option 5) | CUDA events = kernel launches + memory copies. API calls like <code>cudaMalloc</code> , <code>cudaDeviceProp</code> queries are NOT stream events. |

## Q8 True/False

| Statement                                            | T/F | Key Reason                                                                 |
|------------------------------------------------------|-----|----------------------------------------------------------------------------|
| Virtual addresses → physical via page tables         | T   | MMU walks page table tree; TLB caches recent translations                  |
| All warps in block execute same instr simultaneously | F   | Only threads WITHIN a warp are lockstep; different warps are independent   |
| Warps can finish at different times                  | T   | No global sync; <code>__syncthreads</code> only syncs within block         |
| GPU has scheduler to pick warps                      | T   | Warp scheduler selects ready warps each cycle to hide memory latency       |
| PTX is intermediate GPU representation               | T   | JIT compiled by driver to actual ISA at runtime                            |
| GPUs can boot an OS                                  | F   | GPUs have no I/O control, interrupt handling, or OS support infrastructure |

## Q9 MC: SM Occupancy &amp; Divergence Conditions

**Best block config** (1536 thread limit, 8 block limit):

- 1024/blk: 1 block = 1024 threads (1024<1536 but 2×1024>1536; only 1 fits) → **1024 active**
- 64/blk: 8 blocks × 64 = 512 threads → capped by block limit → **512 active**
- 256/blk: 6 blocks × 256 = 1536 → fills SM exactly → **BEST ✓**
- 128/blk: 8 blocks × 128 = 1024 → capped by block limit → **1024 active**

**Which causes warp divergence?**

- `if(threadIdx.x > 4)` → **DIVERGES**. Warp 0 contains threads 0–31. Threshold at 4 splits: threads 0–4 take false branch, threads 5–31 take true branch → divergence.
- `if(blockDim.x > 4)` → NO: blockDim is compile-time constant; same value for all threads.
- `if(blockIdx.x > 4)` → NO: all threads in a block share the same blockIdx.

## GENERATED PRACTICE QUESTIONS — FULLY SOLVED

## EQ1: Thread Indexing in 2D Grid

A kernel uses `blockDim=(32,16)`, `grid=(8,4)`. Thread (7,3) in block (5,2). Global row? col? Linear index into 512×256 image?

**SOLUTION:** row = blockIdx.y × blockDim.y + threadIdx.y = 2×16 + 3 = **35 col** = blockIdx.x × blockDim.x + threadIdx.x = 5×32 + 7 = **167**  
 Linear (row-major) = row × width + col = 35 × 512 + 167 = **18,087**

*Verify bounds: row 35 < 256 ✓, col 167 < 512 ✓. Always check boundary in kernel.*

## EQ2: Occupancy Calculation

SM: 65536 registers, 32768 bytes shared, 2048 max threads. Kernel: 40 regs/thread, 8192 bytes shared/block, 128 threads/block. How many blocks fit? What is occupancy?

**SOLUTION:** • Thread limit: 40/128 = 16 blocks possible • Register limit: 65536/(40×128) = 65536/5120 = 12.8 → **12 blocks** •

Shared mem limit: 32768/8192 = **4 blocks**

**Bottleneck: shared memory (4 blocks)**

Active threads = 4 × 128 = 512. Active warps = 512/32 = 16. Max warps = 2048/32 = 64.

**Occupancy = 16/64 = 25%**

**Fix:** Reduce shared mem per block (smaller tile) → more blocks → better occupancy.

## EQ3: Coalescing Analysis

Warp of 32 threads. Each accesses a float (4B). Pattern A: thread i reads A[i]. Pattern B: thread i reads A[i\*8]. Pattern C: thread i reads A[blockDim.x\*blockIdx.x + i]. How many 128-byte cache line transactions for each?

**SOLUTION:**

A: Threads 0–31 read floats at bytes 0,4,8,...,124 → span = 128 bytes = exactly 1 cache line. **1 transaction ✓**

B: Thread i reads A[i\*8]. Threads 0–31 read addresses 0,32,64,...,992 bytes. Span = 992 bytes. Cache lines: each 128B → 8 transactions. But actually each is isolated: 32 separate accesses to 32 different cache lines → **32 transactions ✗**

C: Same as A (just offset by block): threads read consecutive floats from start of block's region → 1 cache line. **1 transaction ✓**

## EQ4: Bank Conflict Analysis

Shared mem has 32 banks, 4 bytes wide. 32 threads access: (a) s[tid], (b) s[2\*tid], (c) s[16\*tid], (d) s[32\*tid]. Which have conflicts?

**SOLUTION:** Bank# = (word\_address) % 32 where word\_address = byte\_addr/4.

(a) s[tid]: word\_addr = tid. Bank# = tid % 32 → threads 0–31 hit banks 0–31 uniquely. **No conflict ✓**

(b) s[2\*tid]: word\_addr = 2\*tid. Bank# = 2\*tid % 32. Threads 0,16 both hit bank 0; threads 1,17 both hit bank 1... **2-way conflict (16 conflicts)**

(c) s[16\*tid]: word\_addr=16\*tid. Bank# = 16\*tid % 32 = (0,16,0,16,...) alternating. **16-way conflict (threads 0,2,4,...,30 all hit bank 0)**

(d) s[32\*tid]: Bank# = 32\*tid % 32 = 0 for all threads. **32-way conflict — worst case!**

## EQ5: Race Condition Detection

Is this kernel correct? All threads write to global then read neighbor:

```
__shared__ float s[256];
s[tid] = global[gid];           // Line 1
float x = s[(tid+1)%256];       // Line 2 — reads neighbor
global[gid] = x;
```

**SOLUTION: INCORRECT — race condition.** Thread tid reads s[tid+1], which was written by thread tid+1. But thread tid+1 may not have executed Line 1 yet at the time tid executes Line 2. Warps execute independently; no ordering guarantee without sync.

**Fix:** Insert `__syncthreads()`; between Line 1 and Line 2. This barrier ensures ALL threads finish writing before ANY thread reads.

## EQ6: Efficient vs Basic Reduction Divergence

Block of 1024 threads. At step s=16, compare: A) `if(tid%(2*16)==0)` vs B) `if(tid<16)`. How many warps diverge in each? Which is better?

**SOLUTION:**

**A (basic, modulo):** Active threads: 0,32,64,...,992 (multiples of 32). These are the FIRST thread of each warp → 32 warps each have exactly 1 active thread → **32 divergent warps**

**B (efficient, converging):** Active threads: 0,1,...,15. All in warp 0 (threads 0–31). Warp 0: threads 0–15 active, 16–31 inactive → **1 divergent warp**. Warps 1–31: fully inactive → no divergence.

**B is 32× better at this step!** This is why efficient reduction uses converging strides.

## EQ7: Grid Size &amp; Boundary Handling

Array of n=1000 floats, block=256. Write the correct kernel launch and required boundary check.

```
// Host: ceiling division for grid
int nBlocks = (n + 255) / 256; // = 4 blocks (launch 1024 threads)
myKernel<<<nBlocks, 256>>>(d_A, d_B, d_C, n);
```

**// Device: MANDATORY boundary check!**

```
__global__ void myKernel(float*A, float*B, float*C, int n){
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if(i < n) { // threads 1000–1023 skipped!
    C[i] = A[i] + B[i];
  }
}
```

*Without boundary check, threads 1000–1023 access A[1000]...A[1023] → out of bounds → undefined behavior / seg fault.*

## EQ8: Matrix Multiply — Memory Access Count

N×N matrix multiply. Without tiling: how many global loads per output element? Total loads for N×N output? With tile size T, how does total change?

**SOLUTION:**

**Without tiling:** Each `C[i][j] = Σ A[i][k]*B[k][j]` for k=0..N-1. Thread loads row i of A (N loads) + col j of B (N loads) = 2N loads per element. Total output = N² elements → **2N² global loads**.

**With tiling (tile=T):** A tile of T×T output elements shares a T×N strip of A and N×T strip of B. Each element in tile: T loads from shared (fast) + 2N/T global loads per tile phase. Total = N×(2N/T)+T×T² = **2N/T² global loads**. T× improvement!

Example: T=16 → 16× fewer global memory transactions.

## EQ9: PTX &amp; Compilation Pipeline

Why does CUDA compile to PTX first instead of directly to GPU machine code? What happens at runtime?

**SOLUTION:** Benefits of PTX intermediate:

- **Forward compatibility:** Code compiled to PTX works on future GPU generations. GPU driver contains JIT compiler that translates PTX to new ISA without user recompiling.
- **Infinite virtual registers:** PTX uses unlimited virtual registers; hardware register allocator runs at JIT time with knowledge of actual register file size.
- **Optimization opportunities:** JIT compiler can apply architecture-specific optimizations at deployment time.
- **Portability:** One binary runs on different GPU models (sm\_70, sm\_80, sm\_86) via different JIT paths.

**Runtime:** GPU driver loads CUBIN (compiled PTX) from executable. If no CUBIN for current GPU, JIT-compiles included PTX (cached to disk after first run). First run: slower due to JIT; subsequent runs: use cached binary.

**EQ10: Pinned Memory & DMA Deep Dive**  
Why is pinned memory required for async CUDA memory transfers? What does the OS do differently with pinned vs regular memory?  
**SOLUTION:**  
**Regular (pageable) memory:** OS can swap physical pages to disk at any time. DMA engine needs stable physical addresses during transfer. If OS swaps the page mid-transfer → DMA reads wrong data / crashes.  
**Pinned (page-locked) memory:** OS marks pages as non-swappable. Physical address guaranteed stable for DMA duration. GPU DMA engine can directly access host RAM without CPU involvement.  
**Result:** With pinned mem, GPU can start async DMA transfer, return control to CPU immediately ( cudaMemcpyAsync ), and CPU + GPU run concurrently. Without pinned: CUDA must first memcpy from pageable → pinned buffer (adds latency, requires sync).  
**Tradeoff:** Pinned memory reduces available RAM for other processes. Use sparingly; allocate once and reuse.

**EQ11: Streams — Concurrency Quiz**  
Which of these achieves concurrent kernel + data transfer? Why?  
A) Single stream: copy H → D then kernel launch  
B) Two streams: copy in stream 1, kernel in stream 2 with regular malloc  
C) Two streams: copy in stream 1 with pinned mem, kernel in stream 2  
**SOLUTION:**  
A: No. Same stream = sequential FIFO. Copy finishes, then kernel starts.  
B: No. Async copy requires pinned memory. Without it, CUDA internally syncs before transfer → no true async.  
C: **YES ✓** Two streams + pinned memory = true overlap. Stream 1's DMA engine and stream 2's GPU compute engine run on different hardware → can operate simultaneously. GPU has separate copy engine(s) and compute engine(s).